

# Regression for Yes/No Outcomes: Logit

MKT 566 · Fall 2026

Davide Proserpio

USC Marshall School of Business

# Before We Start

# Housekeeping

---

- **Homework 2** (box office, OLS and logit) is out, due **Thursday, Oct. 8**. Everything it needs is in Thursday's deck and today's
- **Group project**: proposal presentations on **Oct. 13 and 15**
- Today: **logit** for about 40 minutes, then the **Airbnb case** on your laptop (aim to finish Task 5 in class; report due **Sunday, Sept. 27**)
- Thursday: **clustering**

# From “how much” to “whether”

---

Thursday’s regressions predicted a **number**: sales, price. Many marketing questions are **yes or no**:

- Will this customer **buy**? **Churn**? **Click** the ad? **Open** the email?
- Will this loan **default**? Will this listing be a **hit**?

## Today you learn to

1. See why a straight line is the wrong tool for a 0/1 outcome
2. Read a **logit** model: sign, odds, and predicted probabilities
3. Run it in R with `glm()` and put it in a **stargazer** table

## What stays the same

- Predictors, dummies, base levels, stars, “holding the others fixed”: all as in OLS
- The AI writes the code. You choose the model and read the result

# R script for today

---

In the [code/](#) folder on the course website, next to Thursday's:

1. [w4-2-logit-class.R](#): every chart and model in today's deck
2. [data/airbnb.csv](#): the same Airbnb listings

[w4-code.zip](#) has all of week 4: Thursday's two scripts, today's, and the data. If you downloaded it on Thursday, you already have everything.

# A Yes/No Outcome

# Our outcome: is a listing a “gem”?

An investor wants listings that are **both** well rated **and** actually booked. Define a **gem** as a listing with a rating of **4.5 or more** and **more than 20 reviews**:

```
1 airbnb[, gem := as.integer(star_rating >= 4.5 & reviews_count > 20)]
2 table(airbnb$gem)
```

```
      0      1
40460 10358
```

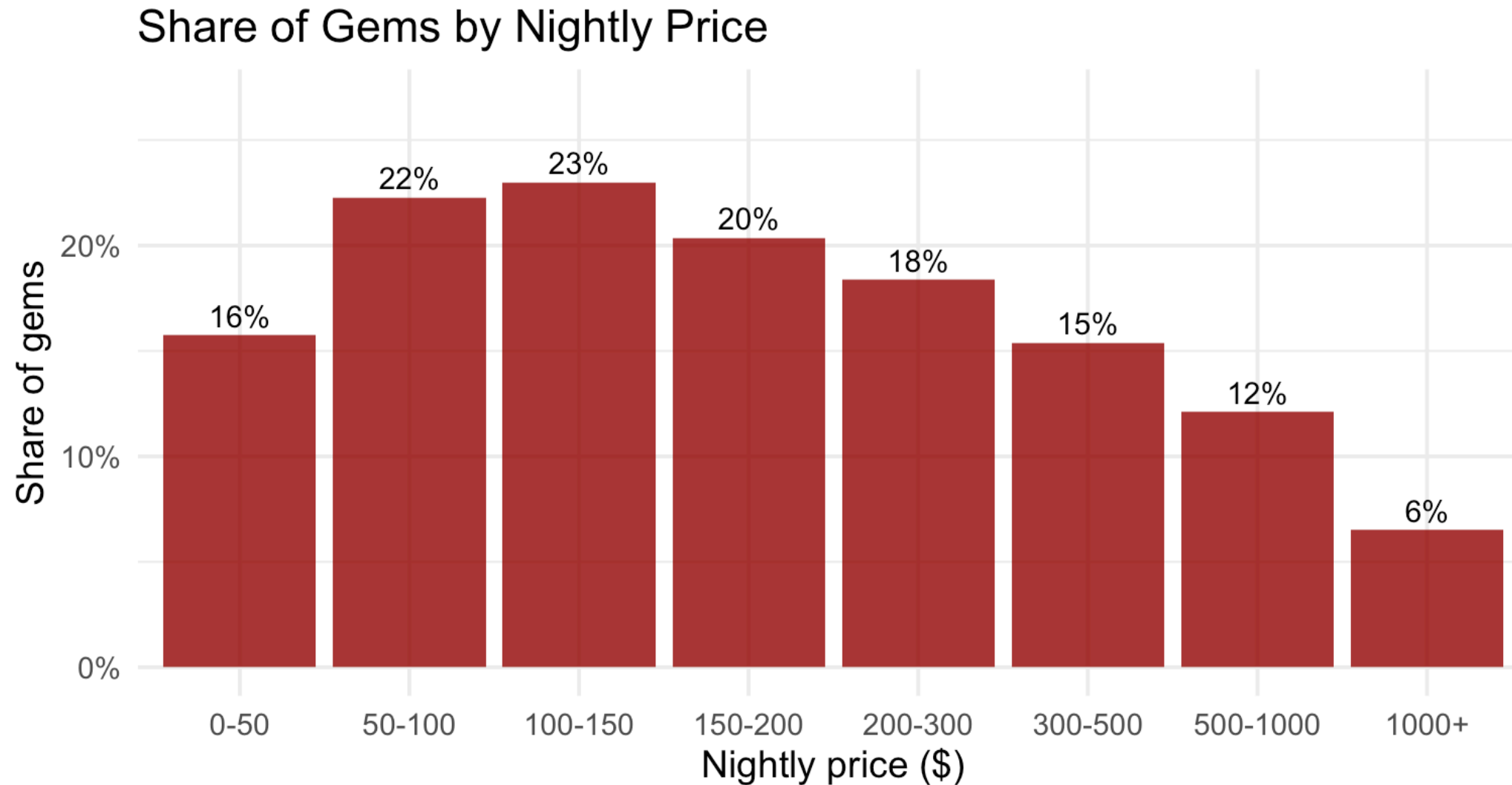
```
1 mean(airbnb$gem)
```

```
[1] 0.2038254
```

- **gem** is a **dummy**: 1 for a gem, 0 otherwise. About **20%** of listings are gems
- The question: **what makes a listing more likely to be a gem?** Start with price

The **&** means both conditions must hold; **as.integer()** turns TRUE/FALSE into 1/0, as in Task 7 of the case.

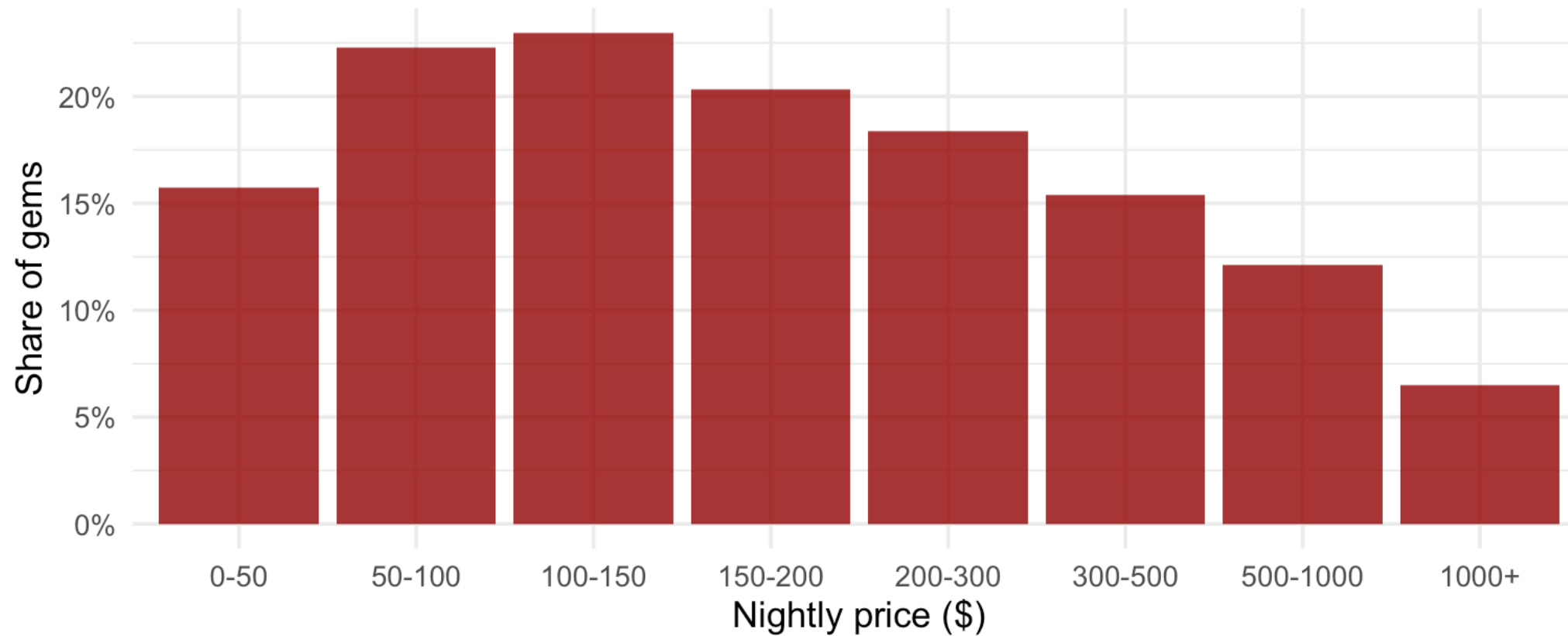
# Share of gems by price



Same idea as the “average by group” charts of week 3, with a 0/1 variable: the average of a dummy is a **share**. What do you see?



# Share of gems by price: reading it

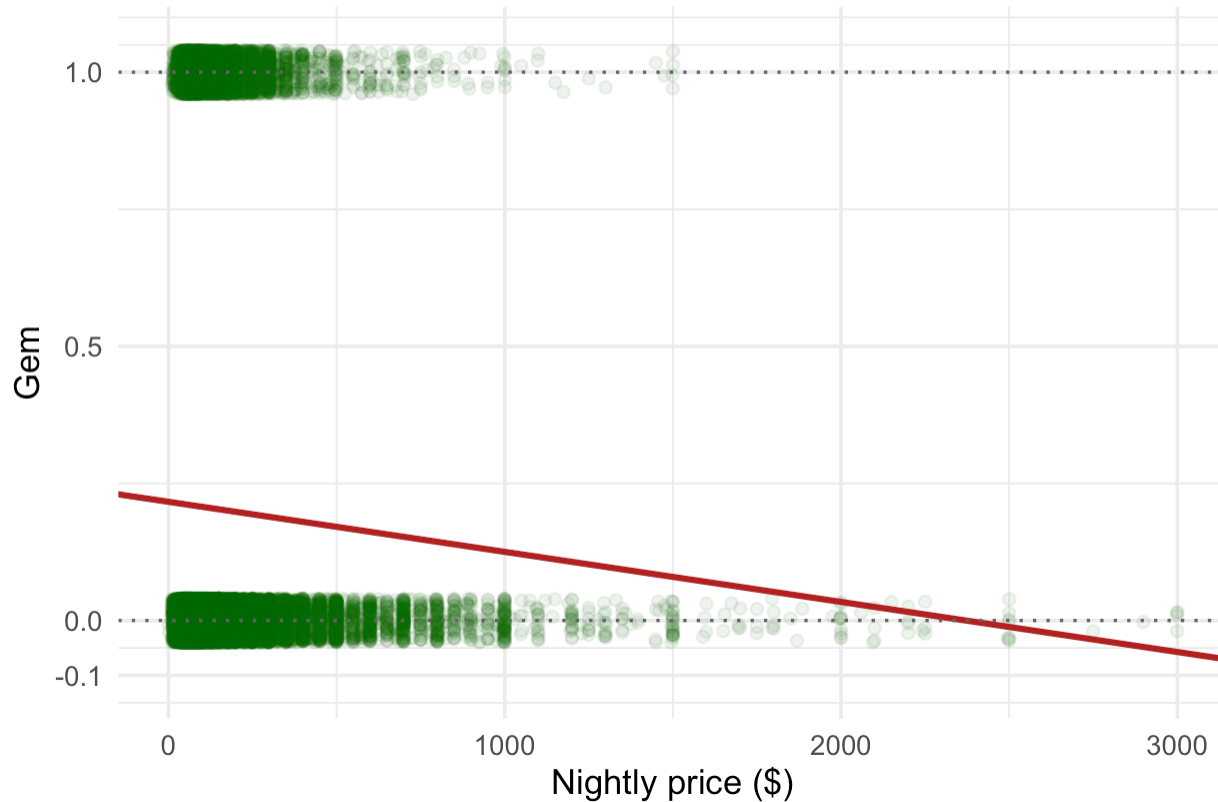


- Gems are **rarer as price rises**: 22–23% of listings between \$50 and \$150, 12% between \$500 and \$1,000, 6% above \$1,000. The very cheapest bin is also lower
- So the **probability** of being a gem depends on price. Can we model that probability with a regression, the way we modeled price on Thursday?

# Try the tool we have: a straight line

```
1 lpm <- lm(gem ~ price, data = airbnb) # OLS, with a 0/1 outcome
```

Gem (0/1) vs. Price, with the OLS line



- The “linear probability model”: the line is the **predicted probability** of being a gem
- Intercept **0.217**: a free listing has a 21.7% chance. Slope **-0.00009**: each extra dollar lowers it by 0.009 percentage points; \$100 more, by about **0.9 points**
- Reasonable so far. Now look at the right end of the line

# Where the line breaks

```
1 pred <- predict(lpm)
2 sum(pred < 0)           # listings with a predicted probability below zero
```

```
[1] 22
```

```
1 round(min(pred), 2)     # the lowest one
```

```
[1] -0.7
```

- The line crosses zero at a price of about **\$2,372** and keeps going: a \$10,000 listing gets a “probability” of **-70%**
- A probability is trapped **between 0 and 1**. As it approaches either edge, the true relationship must **flatten**. A straight line cannot flatten: it falls at the same rate forever
- The slope has the same flaw: the line takes **0.9 points** off for every \$100, whether a listing’s chance of being a gem is at 20% (a cheap listing) or at 2% (a very expensive one). At 2% there are only 2 points left to lose, so near the edge the effect of price must shrink

For a yes/no outcome we need a curve that is **linear in the middle** and **flattens at both ends**. That curve is the logistic function, and the model built on it is the **logit** (logistic regression).

# Logistic Regression

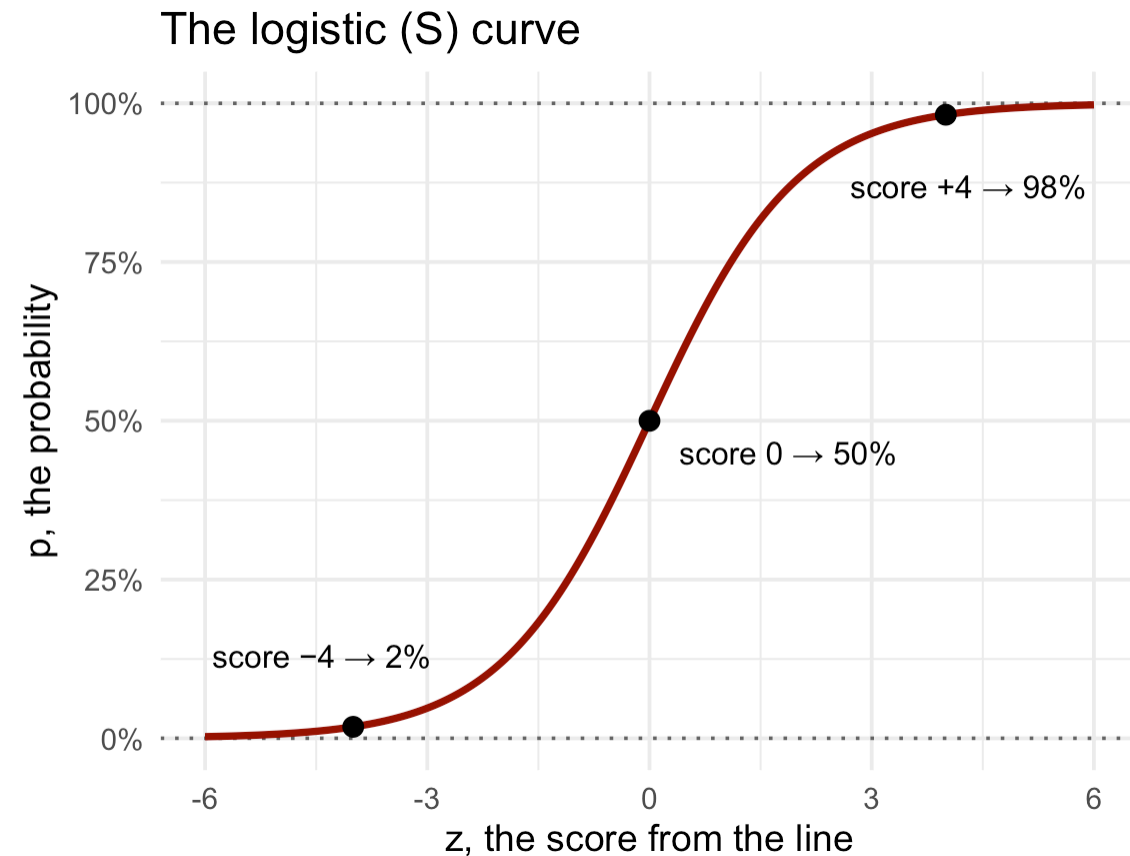
# The fix: squash the line into 0–1

Two steps, and only the second one is new:

1. Keep Thursday's straight line, but treat its output as a **score**, not as the probability:  
 $z = \beta_0 + \beta_1 X$ . A score can be any number:  
0.3, 12, -70

2. **Squash** the score into 0–1 with the **logistic function**:  $p = \frac{1}{1 + e^{-z}}$ . Any score in, a probability out

- A huge negative score becomes a probability near 0, **never below it**; a huge positive one, near 1, never above
- You will never compute this by hand: R does. Your job is to read what comes out



# Odds and log-odds

The **odds** are the chance it happens, divided by the chance it does not. Sports betting already speaks this language:

| Probability                                       | Odds = $p/(1 - p)$ | In words                                |
|---------------------------------------------------|--------------------|-----------------------------------------|
| $p = 20\%$ (the chance a random listing is a gem) | $0.2/0.8 = 0.25$   | one to four: 1 gem for every 4 non-gems |
| $p = 50\%$                                        | $0.5/0.5 = 1$      | even: as likely as not                  |
| $p = 80\%$                                        | $0.8/0.2 = 4$      | four to one: 4 gems for every non-gem   |

- Rearranging the S-curve puts the model on this scale:  $\frac{p}{1 - p} = e^z$ , and taking logs,  
$$\log\left(\frac{p}{1 - p}\right) = z = \beta_0 + \beta_1 X$$
- The right-hand side is Thursday's **straight line**. That is what “logit” means, and why everything from OLS carries over: predictors, dummies, base levels, standard errors, stars

A logit coefficient  $\beta_1$  is the change in the **log-odds** for one more unit of  $X$ . Nobody thinks in log-odds, so the next slide converts.

# Reading a logit coefficient

The rule: one more unit of  $X$  **multiplies the odds by**  $e^{\beta_1}$ ; R computes it for you (`exp(coef(m))`). The worked example previews the model we run next, a logit of **gem on price**: its coefficient will come out at  $\beta_1 = -0.0008$  per dollar. Your job is three steps:

| Step                                                                                               | Price example: $\beta_1 = -0.0008$ per dollar                                                                            |
|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| 1. <b>Sign and stars</b> , as in OLS                                                               | negative, significant: pricier listings are less likely to be gems                                                       |
| 2. <b>Scale</b> : multiply $\beta_1$ by the change you care about                                  | for \$100 more: $100 \times -0.0008 = -0.08$                                                                             |
| 3. <b>Convert</b> : odds are multiplied by $e^{\beta_1}$ ; the percent change is $e^{\beta_1} - 1$ | $e^{-0.08} = 0.923$ : \$100 more multiplies the odds by 0.923, and $0.923 - 1 = -0.077$ : a <b>7.7% drop in the odds</b> |

- Shortcut, as in Thursday’s log-level model: while the scaled coefficient is within about  $\pm 0.2$ , just move the decimal ( $-0.08 \rightarrow$  about  $-8\%$ ); bigger than that, use `exp()`

The 8% drop is a change in the **odds**, not in the probability of being a gem. What it does to the probability takes one more conversion: next slide.

# Odds fall 8%: what happens to the probability?

Not “minus 8 points”. The 8% lives on the **odds** scale, so the route has three steps: go to odds, apply the change there, come back.

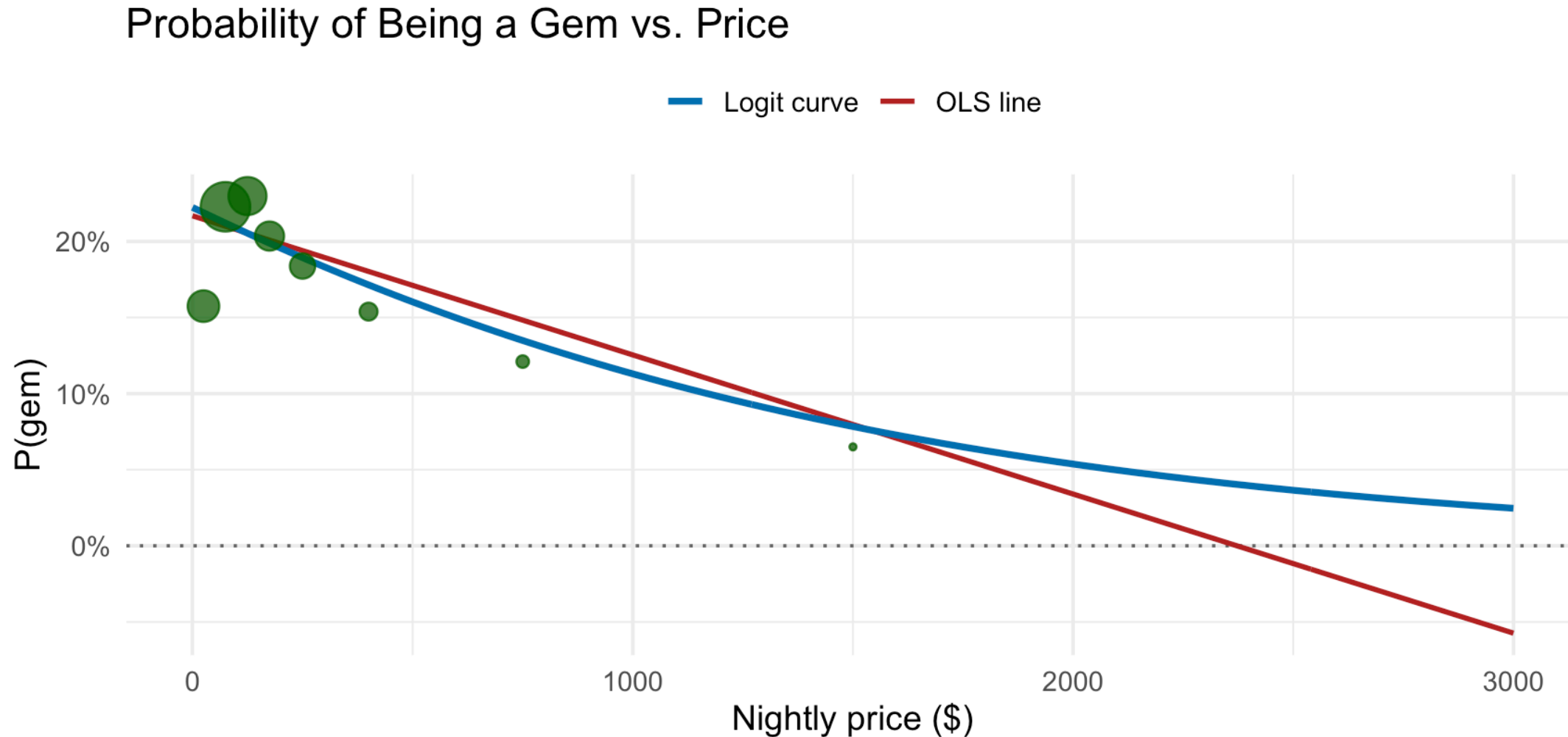
| Step                            | Rule                                     | Our numbers                                                                                  |
|---------------------------------|------------------------------------------|----------------------------------------------------------------------------------------------|
| 1. Probability → odds           | $\text{odds} = p/(1 - p)$                | a typical listing is a gem with $p = 20\%$ (slide 14’s table):<br>$0.20/0.80 = 0.25$         |
| 2. Apply the change to the odds | multiply by $e^{\text{scaled } \beta_1}$ | \$100 more multiplies the odds by 0.923 (last slide, step 3):<br>$0.25 \times 0.923 = 0.231$ |
| 3. Odds → probability           | $p = \text{odds}/(1 + \text{odds})$      | $0.231/1.231 = 18.8\%$                                                                       |

**The whole trip:** 20% → odds 0.25 →  $\times 0.923$  → odds 0.231 → back to **18.8%**

\$100 more takes a typical listing from 20% to **18.8%**. The **odds** fell 7.7% (the “8%”); the **probability** fell **1.2 percentage points**. Same change, two scales, and the probability always moves by less than the odds.



# Logit on price: the curve, on the data



Green dots: the observed share of gems in each price bin (sized by number of listings). Which model fits the dots, and what happens past \$2,000?

# The curve: reading it

```
1 lg1 <- glm(gem ~ price, data = airbnb,  
2           family = binomial)  
3 stargazer(lg1, type = "text",  
4           omit.stat = c("f", "ser", "aic", "bic"),  
5           digits = 4, no.space = TRUE)
```

| Dependent variable: |                        |
|---------------------|------------------------|
| Gem (0/1)           |                        |
| Price               | -0.0008***<br>(0.0001) |
| Constant            | -1.2520***<br>(0.0163) |
| Observations        | 50,818                 |
| Log Likelihood      | -25,650                |

- Every regression you report goes in a **table**, logit included: same `stargazer` call as Thursday
- The curve never dips below zero: at \$2,372, where the line hits zero, it still says about **4%**; at \$10,000, essentially zero
- The coefficient, with the recipe. **Sign and stars:** negative, significant. **Scale**, for \$100:  
 $100 \times -0.0008 = -0.08$
- **Convert:** the shortcut says about an **8% drop in the odds** per \$100; the exact number,  $e^{-0.08} - 1 = -7.7\%$ , agrees

Odds are for reading the table. What a manager wants is the **predicted probability**: the chance that a listing at a given price is a gem. Next slide: how to get it, at any price you like.

# A probability at any price

The two steps from “the fix” slide, run forward with the two numbers in the table: **score, then squash**.

1. **Score**. Take the constant and the price coefficient from the table. At \$500:

$$z = -1.252 + (-0.0008 \times 500) = -1.65$$

2. **Squash**.  $p = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{1.65}} = 0.16$ : a \$500 listing has a **16%** chance of being a gem

`predict()` runs both steps for you. Give it the **saved model** (`lg1`, which remembers the two coefficients), the prices you want as a little table, and `type = "response"`, which means “answer in probabilities”:

```
1 prices <- data.frame(price = c(50, 500, 5000))
2 round(predict(lg1, prices, type = "response"), 3)
```

```
      1      2      3
0.215 0.160 0.005
```

One probability per price: **21.5%** at \$50, **16%** at \$500 (matching the hand computation above), **0.5%** at \$5,000. The blue curve is exactly these numbers, computed at every price.

# How it is estimated, and how to judge the fit

---

**Estimation.** OLS picks the line with the smallest squared errors. Logit picks the coefficients that make the observed 0s and 1s **most likely** (**maximum likelihood**). R does it in a fraction of a second; you never need the formula.

**Log-likelihood.** How likely the data is under the model. Always negative; **closer to zero is better**. Use it to compare models on the same data, the way you used  $R^2$  on Thursday.

For the price-only model: log-likelihood  $-25,650$  vs.  $-25,697$  without price; pseudo- $R^2$   $0.002$ . Price matters, and explains very little, the same lesson as Thursday.

**There is no  $R^2$ .** Alternatives:

- **Pseudo- $R^2$**  (McFadden): how much the log-likelihood improves over a model with no predictors. 0 to 1, but values of 0.1–0.3 are already decent
- **Accuracy:** predict “gem” when  $p > 0.5$  and count the hits. Beware the base rate: saying “not a gem” for **everyone** is right **80%** of the time here. Week 9 (classifiers) covers this properly

# Logit in R

# One function: glm()

glm() works like lm(), with family = binomial to ask for a logit:

```
1 m_logit <- glm(gem ~ price + guests_included + city + room_type, data = airbnb, family = binomial)
2 stargazer(m_logit, type = "text", omit.stat = c("f", "ser", "aic", "bic"),
3           digits = 4, no.space = TRUE)
```

|                 | Dependent variable:    |
|-----------------|------------------------|
|                 | Gem (0/1)              |
| Price           | -0.0017***<br>(0.0001) |
| Guests included | 0.1119***<br>(0.0079)  |
| Boston          | 0.1674***<br>(0.0405)  |
| Los Angeles     | 0.1617***<br>(0.0293)  |
| Miami           | -0.1955***<br>(0.0416) |
| Private room    | -0.0072<br>(0.0265)    |
| Shared room     | -0.7068***<br>(0.0642) |
| Constant        | -1.3935***<br>(0.0349) |
| Observations    | 50,818                 |
| Log Likelihood  | -25,420                |

Same table, typeset for the slide; in your terminal it prints as text. Standard errors in parentheses under each coefficient; stars: \*p < 0.1, \*\*p < 0.05, \*\*\*p < 0.01. Read the signs, then the sizes. Which predictor has the largest effect?

# Reading the table

| Coefficient                         | Odds multiply by             | Reading (holding the others fixed)               |
|-------------------------------------|------------------------------|--------------------------------------------------|
| Price −0.0017                       | $e^{-0.17} = 0.84$ per \$100 | \$100 more: odds of being a gem <b>16% lower</b> |
| Guests included 0.11                | 1.12 per guest               | one more guest included: odds <b>12% higher</b>  |
| Boston 0.17, Los Angeles 0.16       | 1.18, 1.18                   | vs. Austin: odds about <b>18% higher</b>         |
| Miami −0.20                         | 0.82                         | vs. Austin: odds <b>18% lower</b>                |
| Private room −0.01, not significant | 0.99                         | same odds as an entire home                      |
| Shared room −0.71                   | 0.49                         | odds <b>halved</b> vs. an entire home            |

- The **largest** effect is the shared-room dummy, not price: a \$100 price difference moves the odds less than the room type does
- Log-likelihood −25,420, vs. −25,697 with no predictors

# From odds to probabilities

Odds ratios compare. Probabilities decide. `predict()` with `type = "response"` gives the probability for any listing you describe:

```
1 examples <- data.frame(price = c(100, 100, 300, 100), guests_included = 2,  
2                           city = c("Austin", "Boston", "Austin", "Austin"),  
3                           room_type = c("Entire home", "Entire home", "Entire home", "Shared room"))  
4 examples$p_gem <- round(predict(m_logit, examples, type = "response"), 3)  
5 examples
```

|   | price | guests_included | city   | room_type   | p_gem |
|---|-------|-----------------|--------|-------------|-------|
| 1 | 100   | 2               | Austin | Entire home | 0.208 |
| 2 | 100   | 2               | Boston | Entire home | 0.236 |
| 3 | 300   | 2               | Austin | Entire home | 0.157 |
| 4 | 100   | 2               | Austin | Shared room | 0.114 |

- An entire home in Austin at \$100 for 2 guests: **21%** chance of being a gem. The same listing in Boston: **24%**. At \$300 instead of \$100: **16%**. As a shared room: **11%**
- This is the sentence for the investor: “moving the same property from Austin to Boston raises the chance it becomes a gem from 21% to 24%”

Odds ratios for a table, probabilities for a decision. Always compute a few predicted probabilities for realistic cases before you write a recommendation.



# OLS and logit, side by side

|                 | (Thursday)                                                                                                        | (today)                                                         |
|-----------------|-------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Outcome         | a number (price, sales)                                                                                           | yes/no (gem, buy, churn, click)                                 |
| Model           | $Y = \beta_0 + \beta_1 X$                                                                                         | $\log \frac{p}{1-p} = \beta_0 + \beta_1 X$                      |
| R               | <code>lm(y ~ x)</code>                                                                                            | <code>glm(y ~ x, family = binomial)</code>                      |
| $\beta_1$ means | one more unit of $X \rightarrow \beta_1$ more units of $Y$                                                        | one more unit of $X \rightarrow \text{odds} \times e^{\beta_1}$ |
| Predictions     | <code>predict(m)</code>                                                                                           | <code>predict(m, type = "response")</code> gives probabilities  |
| Fit             | $R^2$ , residual standard error                                                                                   | log-likelihood, pseudo- $R^2$                                   |
| Estimation      | least squares                                                                                                     | maximum likelihood                                              |
| Same in both    | predictors, logs of $X$ , dummies and base levels, SE, stars, “holding the others fixed”,<br><b>not causation</b> |                                                                 |

# Still not causation

---

Cheaper listings are more likely to be gems. Three stories, as always:

1. **Price causes gem status**: lower price → more bookings → more reviews → gem
2. **The host causes both**: a host who hustles sets a sharp price *and* cleans well, answers fast, earns 5 stars; a careless host overprices *and* under-delivers. The low price is a symptom of a good host, not a lever anyone can pull
3. **Age causes both**: gems are **older** (20+ reviews take time), and older listings were priced in an earlier, cheaper market

“A \$100 higher price is **associated with** 16% lower odds of being a gem.” The model holds fixed guests, city, and room type. It does not hold fixed listing age, photos, or host effort.

# The Airbnb Case

# The case

---

The handout ([w4-airbnb-case.html](#)) works like RateBeer, with less help: an exact prompt for Task 1, skeletons for Tasks 2–3, and only the goal from Task 4 on.

The client is an **investor** deciding where to buy and what to list:

1. **Tasks 1–3**: look at the data, price vs. reviews with and without logs, the three models side by side (Thursday, applied)
2. **Tasks 4–5**: more predictors, then city as a factor and as hand-made dummies
3. **Tasks 6–7**: a revenue proxy, room type, a quality dummy
4. **Task 8**: three recommendations, one of them a limitation

Under every task: **predict** before you run, **explain back** one line after.

Aim to finish **Task 5** in class. Deliverable, same as always: an **R Markdown report knitted to HTML** plus [ai-log.md](#), due **Sunday, Sept. 27, 11:59 pm**. Data: [airbnb-case-data.zip](#).

# Wrap-up

---

Code and data to reproduce today's models, on the course website:

- [w4-code.zip](#): both week 4 scripts and the data
- [w4-2-logit-class.R](#): every chart and model shown today
- [The Airbnb case handout](#) and its [data](#)

Required reading: Chapter [3.6](#) of *R for Marketing Students*. Optional: Chapter 9.2 of *R for Marketing Research and Analytics*.

# Questions?